

# Stale Hints Cleanup Procedure

---

## Overview

---

This document provides a comprehensive test procedure and operational guide for understanding, identifying, and cleaning up stale hints files in Cassandra clusters running on Kubernetes with Mission Control.

## Table of Contents

---

1. [Background: How Hints Work](#)
2. [The Stale Hints Problem](#)
3. [Auto-Recovery Behavior](#)
4. [Test Procedure](#)
5. [Cleanup Methods](#)
6. [Production Recommendations](#)

## Background: How Hints Work

---

### Hinted Handoff Mechanism

When a coordinator node receives a write but a replica node is unavailable:

1. **Hint Creation:** Coordinator stores a "hint" (the pending mutation) to local disk
2. **Storage Location:** `$CASSANDRA_DATA/hints/<target-node-uuid>/`
3. **Hint Window:** Hints stored for `max_hint_window_in_ms` (default: 3 hours)
4. **Delivery:** When target node recovers, coordinator replays hints in background
5. **Cleanup:** Hints deleted after successful delivery

### Node UUID Creation

- **When:** Generated on first node startup via `UUID.randomUUID()`
- **Storage:** Persisted in `system.local` table and PersistentVolume

- **Lifetime:** Never changes for the node (survives restarts)
- **Visibility:** Broadcast via gossip, stored in `system.peers` on all nodes

## The Stale Hints Problem

---

### What Are Stale Hints?

Stale hints are hint files that will never be delivered because:

1. **Node Replaced:** Target node replaced with new UUID (old UUID no longer exists)
2. **Node Permanently Down:** Node never recovered and never removed from cluster
3. **Node Removed:** Node removed but hints not cleaned up

### Critical Issue

**Cassandra has NO automatic mechanism to clean up hints for nodes that are permanently gone or replaced.**

### Consequences

- **Disk Space Waste:** Hints consume space indefinitely
- **Operational Confusion:** Metrics show pending hints that will never deliver
- **Log Spam:** Failed delivery attempts logged repeatedly
- **Potential Disk Exhaustion:** Multiple failed nodes over time accumulate stale hints

## Auto-Recovery Behavior

---

### Scenario: 15-Node Cluster, 4 Nodes Go Down

When 4 nodes fail at various times during the day, here's what happens:

### cass-operator Behavior

**What It DOES Automatically:**

-  **Pod Restart:** Attempts to restart failed pods automatically
-  **Same UUID:** Restarted pod uses same PersistentVolume and UUID
-  **StatefulSet Reconciliation:** Ensures desired replica count maintained
-  **Health Checks:** Monitors pod readiness and liveness

### What It DOES NOT Do:

-  **Node Replacement:** Does NOT automatically replace with new UUID
-  **replace\_address:** Does NOT perform replace\_address operations
-  **Hint Cleanup:** Does NOT clean up stale hints
-  **PV Deletion:** Does NOT delete PersistentVolumes

### What the Operator Sees:

```
# Pod status shows failures
kubectl get pods -n cassandra-ns
```

bash

| NAME                  | READY | STATUS           | RESTARTS | AGE |          |
|-----------------------|-------|------------------|----------|-----|----------|
| cluster1-dc1-rack1-0  | 2/2   | Running          | 0        | 5d  |          |
| cluster1-dc1-rack1-1  | 0/2   | CrashLoopBackOff | 5        | 2h  | # Failed |
| cluster1-dc1-rack1-2  | 2/2   | Running          | 0        | 5d  |          |
| cluster1-dc1-rack1-3  | 0/2   | Error            | 0        | 4h  | # Failed |
| ...                   |       |                  |          |     |          |
| cluster1-dc1-rack1-10 | 0/2   | Pending          | 0        | 1h  | # Failed |
| cluster1-dc1-rack1-14 | 0/2   | ImagePullBackOff | 0        | 30m | # Failed |

### Alerts/Errors Generated:

- Kubernetes events: Pod failures, restart loops
- cass-operator logs: Reconciliation failures
- Prometheus alerts (if configured): PodNotReady, CassandraNodeDown

## Mission Control Behavior

### What It DOES Automatically:

-  **Monitoring:** Detects node failures via health checks
-  **Alerting:** Sends alerts/notifications for down nodes
-  **Visibility:** Dashboard shows node status and metrics
-  **Metrics Collection:** Continues collecting available metrics

## What It DOES NOT Do:

- **✗ Auto-Replacement:** Does not automatically replace nodes
- **✗ Auto-Repair:** Does not trigger repair automatically
- **✗ Hint Cleanup:** Does not clean up stale hints
- **✗ Topology Changes:** Does not modify cluster topology

## What the Operator Sees:

- Mission Control Dashboard: 4 nodes marked as DOWN/UNREACHABLE
- Alerts: Email/Slack notifications for node failures
- Metrics: Degraded cluster health score
- Recommendations: May suggest actions but doesn't execute them

## Automated Recovery Actions

### What IS Automated:

1. **Pod Restart Attempts:** Kubernetes/cass-operator will keep trying to restart pods
2. **If PV is intact and pod can start:** Node comes back with SAME UUID (no hints issue)
3. **Health Monitoring:** Continuous monitoring and alerting

### What IS NOT Automated:

1. **Node Replacement:** Requires manual `replace_address` operation
2. **PV Cleanup:** Requires manual PVC deletion for new UUID
3. **Hint Cleanup:** Requires manual or scripted cleanup
4. **Topology Changes:** Requires manual `removenode` or scaling decisions

**Key Takeaway:** Both systems will attempt to restart pods but won't perform `replace_address` operations or clean up stale hints automatically. **Operator intervention is required for true node replacement.**

## Test Procedure

---

# Practical Operational Guide: Running on 14 Nodes After Node Failure

## Scenario: Node Down 5+ Hours, Decision to Run on 14 Nodes

When a node has been down for 5+ hours and you decide NOT to bring it back up:

### Step 1: Identify the Down Node's UUID

#### Method 1: From Any Running Node (Recommended)

```
# Get cluster status from any running pod
kubectl exec -n cassandra-ns cluster1-dc1-rack1-0 -c cassandra -- nodetool status
```

*# Output shows ALL nodes including down ones with their UUIDs*

Datacenter: dc1

|/ State=Normal/Leaving/Joining/Moving

| --  | Address    | Load  | Host ID                               | Rack  |             |
|-----|------------|-------|---------------------------------------|-------|-------------|
| UN  | 10.244.1.1 | 1.5TB | aaaa-1111-bbbb-2222-cccc-333344445555 | rack1 |             |
| DN  | 10.244.1.5 | 1.4TB | eeee-5555-ffff-6666-gggg-777788889999 | rack1 | # DOWN node |
| UN  | 10.244.1.3 | 1.6TB | cccc-3333-dddd-4444-eeee-555566667777 | rack1 |             |
| ... |            |       |                                       |       |             |

bash

The UUID is in the "Host ID" column for the DN (Down) node.

#### Method 2: Query System Tables

```
# From any running node, query system.peers
kubectl exec -n cassandra-ns cluster1-dc1-rack1-0 -c cassandra -- cqlsh -e "
SELECT peer, host_id, rack, data_center
FROM system.peers
WHERE peer = '10.244.1.5';
"
```

*# Output:*

| # peer                    | host_id                               | rack  | data_center |
|---------------------------|---------------------------------------|-------|-------------|
| # -----+-----+-----+----- |                                       |       |             |
| # 10.244.1.5              | eeee-5555-ffff-6666-gggg-777788889999 | rack1 | dc1         |

bash

#### Method 3: From Mission Control

1. Navigate to Mission Control Dashboard
2. Go to Cluster → Nodes
3. Find the down node (marked with red/warning indicator)
4. Click on the node details

## 5. Copy the "Host ID" field

### Method 4: From Kubernetes Pod Name to UUID

```
# If you know the pod name (e.g., cluster1-dc1-rack1-4)
# Get the pod's IP address
POD_IP=$(kubectl get pod cluster1-dc1-rack1-4 -n cassandra-ns -o jsonpath='{.status.podIP}')

# Query for that IP's UUID from a running node
kubectl exec -n cassandra-ns cluster1-dc1-rack1-0 -c cassandra -- cqlsh -e "
SELECT peer, host_id FROM system.peers WHERE peer = '$POD_IP';
"
```

### Method 5: Check Existing Hint Directories

```
# List hint directories on any running node
kubectl exec -n cassandra-ns cluster1-dc1-rack1-0 -c cassandra -- \
  ls -la /var/lib/cassandra/data/hints/

# Output shows directories named with UUIDs
drwxr-xr-x 2 cassandra eeee-5555-ffff-6666-gggg-777788889999/ # This is a hint target

# Cross-reference with nodetool status to identify which is the down node
```

## Step 2: Remove the Node from Cluster

```
# Use the UUID from Step 1
DOWN_NODE_UUID="eeee-5555-ffff-6666-gggg-777788889999"

# Remove the node from the cluster
kubectl exec -n cassandra-ns cluster1-dc1-rack1-0 -c cassandra -- \
  nodetool removenode $DOWN_NODE_UUID

# Monitor removal progress
kubectl exec -n cassandra-ns cluster1-dc1-rack1-0 -c cassandra -- \
  nodetool removenode status

# Wait for completion (can take hours depending on data size)
```

## Step 3: Verify Node Removed

```
# Check cluster status - down node should no longer appear
kubectl exec -n cassandra-ns cluster1-dc1-rack1-0 -c cassandra -- nodetool status

# Should now show only 14 nodes
```

## Step 4: Clean Up Stale Hints

```
# Now clean up hints for the removed node on all remaining nodes
DOWN_NODE_UUID="eeee-5555-ffff-6666-gggg-777788889999"

# Get all running pods
RUNNING_PODS=$(kubectl get pods -n cassandra-ns -l app=cassandra \
  --field-selector=status.phase=Running -o name)

# Delete hints on each node
for POD in $RUNNING_PODS; do
  POD_NAME=$(basename $POD)
  echo "Cleaning hints on $POD_NAME..."

  kubectl exec -n cassandra-ns $POD_NAME -c cassandra -- \
    rm -rf /var/lib/cassandra/data/hints/$DOWN_NODE_UUID

  echo "  Done"
done
```

bash

## Step 5: Verify Hints Cleanup

```
# Verify no hints remain for the removed node
for POD in $RUNNING_PODS; do
  POD_NAME=$(basename $POD)
  kubectl exec -n cassandra-ns $POD_NAME -c cassandra -- \
    ls /var/lib/cassandra/data/hints/$DOWN_NODE_UUID 2>&1 | grep -q "No such file" && \
    echo "$POD_NAME: Hints cleaned" || \
    echo "$POD_NAME: Hints still present"
done
```

bash

## Step 6: Update Kubernetes Resources (Optional)

```
# If you want to scale down the StatefulSet to 14 replicas
kubectl scale statefulset cluster1-dc1-rack1 -n cassandra-ns --replicas=14

# Delete the PVC for the removed node
kubectl delete pvc cassandra-data-cluster1-dc1-rack1-4 -n cassandra-ns
```

bash

## Complete Operational Script

bash

```
#!/bin/bash
# Script: remove-down-node-and-cleanup-hints.sh
# Purpose: Remove a permanently down node and clean up stale hints

set -e

NAMESPACE="${1:-cassandra-ns}"
POD_TO_REMOVE="${2}" # e.g., cluster1-dc1-rack1-4

if [ -z "$POD_TO_REMOVE" ]; then
    echo "Usage: $0 <namespace> <pod-name>"
    echo "Example: $0 cassandra-ns cluster1-dc1-rack1-4"
    exit 1
fi

echo "=== Step 1: Identify Down Node UUID ==="
# Get a running pod for commands
RUNNING_POD=$(kubectl get pods -n $NAMESPACE -l app=cassandra \
    --field-selector=status.phase=Running -o name | head -1 | xargs basename)

echo "Using running pod: $RUNNING_POD"

# Get the IP of the pod to remove
POD_IP=$(kubectl get pod $POD_TO_REMOVE -n $NAMESPACE -o jsonpath='{.status.podIP}' 2>/dev/null ||
echo "")

if [ -z "$POD_IP" ]; then
    echo "ERROR: Could not find IP for pod $POD_TO_REMOVE"
    echo "Checking nodetool status for down nodes..."
    kubectl exec -n $NAMESPACE $RUNNING_POD -c cassandra -- nodetool status | grep "^DN"
    exit 1
fi

# Get UUID for that IP
DOWN_NODE_UUID=$(kubectl exec -n $NAMESPACE $RUNNING_POD -c cassandra -- cqlsh -e \
    "SELECT host_id FROM system.peers WHERE peer = '$POD_IP';" | \
    grep -v "host_id" | grep -v "^$" | grep -v "^-" | tr -d ' ')

if [ -z "$DOWN_NODE_UUID" ]; then
    echo "ERROR: Could not find UUID for IP $POD_IP"
    exit 1
fi

echo "Found down node UUID: $DOWN_NODE_UUID"

echo ""
echo "=== Step 2: Remove Node from Cluster ==="
kubectl exec -n $NAMESPACE $RUNNING_POD -c cassandra -- \
    nodetool removenode $DOWN_NODE_UUID

echo "Waiting for removenode to complete..."
while true; do
    STATUS=$(kubectl exec -n $NAMESPACE $RUNNING_POD -c cassandra -- \
        nodetool removenode status 2>&1 || echo "done")

    if echo "$STATUS" | grep -q "No removenode operations"; then
        echo "Removenode complete!"
        break
    fi
done
```



```

echo "  Status: $STATUS"
sleep 30
done

echo ""
echo "=== Step 3: Verify Node Removed ==="
kubectl exec -n $NAMESPACE $RUNNING_POD -c cassandra -- nodetool status

echo ""
echo "=== Step 4: Clean Up Stale Hints ==="
RUNNING_PODS=$(kubectl get pods -n $NAMESPACE -l app=cassandra \
  --field-selector=status.phase=Running -o name)

for POD in $RUNNING_PODS; do
  POD_NAME=$(basename $POD)
  echo "Cleaning hints on $POD_NAME..."

  kubectl exec -n $NAMESPACE $POD_NAME -c cassandra -- \
    rm -rf /var/lib/cassandra/data/hints/$DOWN_NODE_UUID 2>/dev/null || true
done

echo ""
echo "=== Step 5: Verify Cleanup ==="
for POD in $RUNNING_PODS; do
  POD_NAME=$(basename $POD)
  if kubectl exec -n $NAMESPACE $POD_NAME -c cassandra -- \
    ls /var/lib/cassandra/data/hints/$DOWN_NODE_UUID 2>&1 | grep -q "No such file"; then
    echo "  ✓ $POD_NAME: Hints cleaned"
  else
    echo "  ✗ $POD_NAME: Hints still present"
  fi
done

echo ""
echo "=== Complete ==="
echo "Node $POD_TO_REMOVE (UUID: $DOWN_NODE_UUID) has been removed"
echo "Stale hints have been cleaned up"
echo ""
echo "Optional: Scale down StatefulSet and delete PVC:"
echo "  kubectl scale statefulset <name> -n $NAMESPACE --replicas=14"
echo "  kubectl delete pvc cassandra-data-$POD_TO_REMOVE -n $NAMESPACE"

```

## Usage Example

```
# Make script executable
chmod +x remove-down-node-and-cleanup-hints.sh

# Run the script
./remove-down-node-and-cleanup-hints.sh cassandra-ns cluster1-dc1-rack1-4

# Output:
# === Step 1: Identify Down Node UUID ===
# Using running pod: cluster1-dc1-rack1-0
# Found down node UUID: eeee-5555-ffff-6666-gggg-777788889999
#
# === Step 2: Remove Node from Cluster ===
# Waiting for removenode to complete...
# Removenode complete!
#
# === Step 3: Verify Node Removed ===
# [Shows 14 nodes]
#
# === Step 4: Clean Up Stale Hints ===
# Cleaning hints on cluster1-dc1-rack1-0...
# Cleaning hints on cluster1-dc1-rack1-1...
# ...
#
# === Step 5: Verify Cleanup ===
#   ✓ cluster1-dc1-rack1-0: Hints cleaned
#   ✓ cluster1-dc1-rack1-1: Hints cleaned
# ...
#
# === Complete ===
```

## Test Environment

- **Cluster:** 6-node Cassandra cluster on Kubernetes
- **Namespace:** `cass-test`
- **Keyspace:** `test_ks` with RF=3
- **Scenario:** 2 nodes fail, 1 replaced, 1 never replaced

## Phase 1: Initial Setup and Baseline

### Step 1.1: Verify Initial Cluster State

```
# Get all pod names and status
kubectl get pods -n cass-test -l app=cassandra

# Expected: All 6 pods Running
# cluster1-dc1-rack1-0 through cluster1-dc1-rack1-5

# Get cluster status and record all UUIDs
kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- nodetool status
```

bash

## Expected Output:

```
UN  10.244.1.1    100GB  aaaa-1111-... # rack1-0
UN  10.244.1.2    100GB  bbbb-2222-... # rack1-1 (will be Node 101)
UN  10.244.1.3    100GB  cccc-3333-... # rack1-2 (will be Node 102)
UN  10.244.1.4    100GB  dddd-4444-... # rack1-3
UN  10.244.1.5    100GB  eeee-5555-... # rack1-4
UN  10.244.1.6    100GB  ffff-6666-... # rack1-5
```

## Record UUIDs for later:

```
NODE_101_UUID="bbbb-2222-cccc-3333-dddd-444455556666"
NODE_102_UUID="cccc-3333-dddd-4444-eeee-555566667777"
```

bash

## Step 1.2: Verify No Existing Hints

```
# Check hints directory on all nodes
for i in {0..5}; do
  echo "=== Checking cluster1-dc1-rack1-$i ==="
  kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
    ls -lh /var/lib/cassandra/data/hints/ 2>/dev/null || echo "Empty"
done
```

bash

**Expected:** All hints directories empty

## Step 1.3: Create Test Keyspace and Table

```
kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- cqlsh -e "
CREATE KEYSPACE IF NOT EXISTS test_ks
WITH replication = {'class': 'NetworkTopologyStrategy', 'dc1': 3};

CREATE TABLE IF NOT EXISTS test_ks.test_table (
  id uuid PRIMARY KEY,
  data text,
  timestamp timestamp
);
"
```

bash

## Phase 2: Simulate Node Failures

### Step 2.1: Take Down Node 101 (rack1-1)

```
# Force delete pod to simulate failure
kubectl delete pod cluster1-dc1-rack1-1 -n cass-test --force --grace-period=0

# Verify it's down
kubectl get pods -n cass-test | grep rack1-1

# Verify cluster sees it as down
kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- \
  nodetool status | grep DN
```

bash

#### Expected Output:

```
DN  10.244.1.2    100GB    bbbb-2222-... # Node 101 DOWN
```

### Step 2.2: Take Down Node 102 (rack1-2)

```
# Force delete second pod
kubectl delete pod cluster1-dc1-rack1-2 -n cass-test --force --grace-period=0

# Verify both nodes down
kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- \
  nodetool status | grep DN
```

bash

#### Expected Output:

```
DN  10.244.1.2    100GB    bbbb-2222-... # Node 101 DOWN
DN  10.244.1.3    100GB    cccc-3333-... # Node 102 DOWN
```

### Step 2.3: Generate Write Load to Create Hints

```
# Create write load script
cat > /tmp/write_load.sh << 'EOF'
#!/bin/bash
for i in {1..10000}; do
    kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- cqlsh -e "
        INSERT INTO test_ks.test_table (id, data, timestamp)
        VALUES (uuid(), 'test data $i', toTimestamp(now()));
    " 2>/dev/null

    if [ $((i % 100)) -eq 0 ]; then
        echo "Inserted $i rows"
    fi
done
EOF

chmod +x /tmp/write_load.sh
/tmp/write_load.sh
```

**Expected:** Writes succeed (CL=QUORUM with RF=3, 2 nodes down still allows writes)

## Step 2.4: Verify Hints Created

```
# Check for hints on all running nodes
echo "=== Checking for hints to Node 101: $NODE_101_UUID ==="
for i in 0 3 4 5; do
    echo "--- cluster1-dc1-rack1-$i ---"
    kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
        sh -c "if [ -d /var/lib/cassandra/data/hints/$NODE_101_UUID ]; then \
            du -sh /var/lib/cassandra/data/hints/$NODE_101_UUID; \
            else echo 'No hints'; fi"
done

echo ""
echo "=== Checking for hints to Node 102: $NODE_102_UUID ==="
for i in 0 3 4 5; do
    echo "--- cluster1-dc1-rack1-$i ---"
    kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
        sh -c "if [ -d /var/lib/cassandra/data/hints/$NODE_102_UUID ]; then \
            du -sh /var/lib/cassandra/data/hints/$NODE_102_UUID; \
            else echo 'No hints'; fi"
done
```

**Expected:** Multiple nodes show hints directories for both UUIDs with data

## Example Results:

```
Node rack1-0: Has hints for 101 (5MB), Has hints for 102 (4MB)
Node rack1-3: Has hints for 101 (3MB), Has hints for 102 (6MB)
Node rack1-4: Has hints for 102 (2MB)
Node rack1-5: Has hints for 101 (4MB)
```

## Phase 3: Replace Node 101 (Within Hint Window)

### Step 3.1: Prepare Replacement Node

```
# Delete the PVC to ensure new UUID (simulating hardware replacement)
kubectl delete pvc cassandra-data-cluster1-dc1-rack1-1 -n cass-test

# Recreate the pod (cass-operator will recreate automatically, or manually scale)
kubectl scale statefulset cluster1-dc1-rack1 -n cass-test --replicas=6
```

bash

### Step 3.2: Perform Replace Address Operation

```
# Get the old UUID for Node 101
OLD_NODE_101_UUID="$NODE_101_UUID"

# Edit the pod to add replace_address JVM option
kubectl exec -n cass-test cluster1-dc1-rack1-1 -c cassandra -- \
  sh -c "echo 'JVM_OPTS=\"\$JVM_OPTS -Dcassandra.replace_address=10.244.1.2\"' >>
/etc/cassandra/jvm.options"

# Restart the pod
kubectl delete pod cluster1-dc1-rack1-1 -n cass-test

# Monitor bootstrap
kubectl logs -n cass-test cluster1-dc1-rack1-1 -c cassandra -f | grep -i "bootstrap\|streaming"
```

bash

### Step 3.3: Verify New UUID Assigned

```
# Wait for node to be UP
kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- nodetool status

# Get new UUID for Node 101
NEW_NODE_101_UUID=$(kubectl exec -n cass-test cluster1-dc1-rack1-1 -c cassandra -- \
  cqlsh -e "SELECT host_id FROM system.local;" | grep -v "host_id" | grep -v "^$" | tr -d ' ')

echo "Old Node 101 UUID: $OLD_NODE_101_UUID"
echo "New Node 101 UUID: $NEW_NODE_101_UUID"
```

bash

**Expected:** Different UUIDs

### Step 3.4: Check Stale Hints for Old Node 101 UUID

```
# Check if old hints still exist
echo "=== Checking for STALE hints to OLD Node 101: $OLD_NODE_101_UUID ==="
for i in 0 3 4 5; do
  echo "--- cluster1-dc1-rack1-$i ---"
  kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
    sh -c "if [ -d /var/lib/cassandra/data/hints/$OLD_NODE_101_UUID ]; then \
      echo 'STALE HINTS FOUND: '; \
      du -sh /var/lib/cassandra/data/hints/$OLD_NODE_101_UUID; \
      ls -lh /var/lib/cassandra/data/hints/$OLD_NODE_101_UUID/; \
      else echo 'No stale hints'; fi"
done
```

bash

**Expected:** Stale hints still present for old UUID

### Step 3.5: Monitor Hint Delivery Attempts

```
# Check logs for failed delivery attempts
kubectl logs -n cass-test cluster1-dc1-rack1-0 -c cassandra --tail=100 | \
  grep -i "hint\|delivery"
```

bash

**Expected:** Log entries showing failed hint delivery attempts to old UUID

## Phase 3B: Replace Node After Hint Window Expires (Alternative Scenario)

This phase demonstrates what happens when a node is replaced AFTER the hint window expires (3+ hours).

### Step 3B.1: Wait for Hint Window to Expire

```
# Check hint window configuration
kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- \
  grep "max_hint_window_in_ms" /etc/cassandra/cassandra.yaml

# Default: 10800000 ms = 3 hours
# For testing, you can reduce this temporarily

# Wait for hint window to expire (3+ hours from when Node 101 went down)
echo "Waiting for hint window to expire..."
echo "Node 101 went down at: [record timestamp from Step 2.1]"
echo "Hint window expires at: [timestamp + 3 hours]"
```

bash

### Step 3B.2: Verify No NEW Hints Being Created

```

# Record current hint sizes for Node 101
echo "=== Hint sizes BEFORE new writes ==="
for i in 0 3 4 5; do
    kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
        du -sh /var/lib/cassandra/data/hints/$NODE_101_UUID 2>/dev/null || echo "No hints on rack1-$i"
done

# Generate new writes
for i in {1..1000}; do
    kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- cqlsh -e "
        INSERT INTO test_ks.test_table (id, data, timestamp)
        VALUES (uuid(), 'post-window data $i', toTimestamp(now()));
    " 2>/dev/null
done

# Check hint sizes again
echo "=== Hint sizes AFTER new writes ==="
for i in 0 3 4 5; do
    kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
        du -sh /var/lib/cassandra/data/hints/$NODE_101_UUID 2>/dev/null || echo "No hints on rack1-$i"
done

```

bash

**Expected:** Hint sizes UNCHANGED (no new hints after window expires)

**Key Observation:** After hint window expires, Cassandra stops creating NEW hints for the down node, but OLD hints remain on disk.

### Step 3B.3: Replace Node 101 After Hint Window

```

# Delete the PVC to ensure new UUID
kubectl delete pvc cassandra-data-cluster1-dc1-rack1-1 -n cass-test

# Recreate the pod
kubectl scale statefulset cluster1-dc1-rack1 -n cass-test --replicas=6

# Wait for pod to be created
kubectl wait --for=condition=ready pod/cluster1-dc1-rack1-1 -n cass-test --timeout=300s

# Perform replace_address
OLD_NODE_101_UUID="$NODE_101_UUID"

kubectl exec -n cass-test cluster1-dc1-rack1-1 -c cassandra -- \
    sh -c "echo 'JVM_OPTS=\"\$JVM_OPTS -Dcassandra.replace_address=10.244.1.2\"' >>
    /etc/cassandra/jvm.options"

kubectl delete pod cluster1-dc1-rack1-1 -n cass-test

# Monitor bootstrap
kubectl logs -n cass-test cluster1-dc1-rack1-1 -c cassandra -f | grep -i "bootstrap\|streaming"

```

bash

### Step 3B.4: Verify New UUID and Check Stale Hints



```

# Get new UUID
NEW_NODE_101_UUID=$(kubectl exec -n cass-test cluster1-dc1-rack1-1 -c cassandra -- \
  cqlsh -e "SELECT host_id FROM system.local;" | grep -v "host_id" | grep -v "^$" | tr -d ' ')

echo "Old Node 101 UUID: $OLD_NODE_101_UUID"
echo "New Node 101 UUID: $NEW_NODE_101_UUID"

# Check for stale hints
echo "=== Checking for STALE hints to OLD Node 101: $OLD_NODE_101_UUID ==="
for i in 0 3 4 5; do
  echo "--- cluster1-dc1-rack1-$i ---"
  kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
    sh -c "if [ -d /var/lib/cassandra/data/hints/$OLD_NODE_101_UUID ]; then \
      echo 'STALE HINTS FOUND: '; \
      du -sh /var/lib/cassandra/data/hints/$OLD_NODE_101_UUID; \
      ls -lh /var/lib/cassandra/data/hints/$OLD_NODE_101_UUID; \
    else echo 'No stale hints'; fi"
done

```

**Expected:** Stale hints STILL PRESENT for old UUID

**Critical Finding:** Even though the hint window expired and no NEW hints were created, the OLD hints from before the window expired are STILL on disk and will never be delivered.

### Step 3B.5: Compare Behavior: Within vs After Hint Window

#### Replacement WITHIN Hint Window (Phase 3):

- Old hints exist and are actively being created until replacement
- Hint delivery attempts continue to fail
- Stale hints accumulate until manual cleanup

#### Replacement AFTER Hint Window (Phase 3B):

- Old hints exist but NO NEW hints being created
- Hint delivery attempts may have stopped
- Stale hints remain frozen at the size when window expired
- Still requires manual cleanup

**Key Insight:** The hint window only controls when NEW hints stop being created. It does NOT automatically clean up existing hints. In both scenarios, manual cleanup is required.

### Step 3B.6: Measure Stale Hints Impact

```
# Calculate total stale hints across cluster
echo "=== Total Stale Hints for Old Node 101 ==="
TOTAL_SIZE=0
for i in 0 3 4 5; do
    SIZE=$(kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
        du -sb /var/lib/cassandra/data/hints/$OLD_NODE_101_UUID 2>/dev/null | awk '{print $1}' || echo
        "0")
    echo "    rack1-$i: $SIZE bytes"
    TOTAL_SIZE=$((TOTAL_SIZE + SIZE))
done

echo "Total stale hints: $TOTAL_SIZE bytes ($((TOTAL_SIZE / 1024 / 1024)) MB)"
```

**Expected:** Significant disk space consumed by stale hints that will never be delivered

## Phase 4: Monitor Node 102 (Never Replaced)

### Step 4.1: Wait for Hint Window to Expire

```
# Check current time and hint window setting
kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- \
    grep "max_hint_window_in_ms" /etc/cassandra/cassandra.yaml

# Default is 10800000 ms = 3 hours
# For testing, you can temporarily reduce:
# Edit cassandra.yaml: max_hint_window_in_ms: 300000 (5 minutes)
```

### Step 4.2: Generate More Writes After Hint Window

```
# After 3+ hours (or configured window), generate more writes
for i in {1..1000}; do
    kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- cqlsh -e "
        INSERT INTO test_ks.test_table (id, data, timestamp)
        VALUES (uuid(), 'post-window data $i', toTimestamp(now()));
    " 2>/dev/null
done
```

### Step 4.3: Verify No NEW Hints Created for Node 102

```

# Record current hint sizes BEFORE new writes
echo "=== Hint sizes for Node 102 BEFORE new writes ==="
for i in 0 3 4 5; do
    kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
        du -sh /var/lib/cassandra/data/hints/$NODE_102_UUID 2>/dev/null || echo "No hints on rack1-$i"
done

# Generate writes (already done above)

# Check hint sizes again AFTER new writes
echo "=== Hint sizes for Node 102 AFTER new writes ==="
for i in 0 3 4 5; do
    kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
        du -sh /var/lib/cassandra/data/hints/$NODE_102_UUID 2>/dev/null || echo "No hints on rack1-$i"
done

```

bash

**Expected:** Hint sizes unchanged (no new hints after window expires)

## Step 4.4: Verify Stale Hints Persist Indefinitely

```

# Wait several hours/days
# Check hints still exist
echo "=== Checking stale hints persistence (Day 2) ==="
for i in 0 3 4 5; do
    kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
        sh -c "if [ -d /var/lib/cassandra/data/hints/$NODE_102_UUID ]; then \
            echo 'STALE HINTS STILL PRESENT on rack1-$i:.'; \
            du -sh /var/lib/cassandra/data/hints/$NODE_102_UUID; \
            fi"
done

```

bash

**Expected:** Stale hints still present, consuming disk space

## Cleanup Methods

### Method 1: Manual Deletion (Safest, Recommended)

#### Step 1: Identify Stale Hint UUIDs

```

# Get current cluster members
kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- nodetool status | \
    awk '/^[UD][NLJ]/ {print $7}' > /tmp/current_uuids.txt

echo "Current cluster UUIDs:"
cat /tmp/current_uuids.txt

```

bash

## Step 2: Find Stale Hint Directories

```
# Script to identify stale hints across all nodes
cat > /tmp/find_stale_hints.sh << 'EOF'
#!/bin/bash
NAMESPACE="cass-test"
CURRENT_UUIDS_FILE="/tmp/current_uuids.txt"

echo "=== Finding Stale Hints ==="
for i in 0 3 4 5; do
    POD="cluster1-dc1-rack1-$i"
    echo ""
    echo "Checking $POD..."

    # Get hint directories
    HINT_DIRS=$(kubectl exec -n $NAMESPACE $POD -c cassandra -- \
        ls /var/lib/cassandra/data/hints/ 2>/dev/null)

    for UUID in $HINT_DIRS; do
        # Check if UUID is in current cluster
        if ! grep -q "$UUID" "$CURRENT_UUIDS_FILE"; then
            SIZE=$(kubectl exec -n $NAMESPACE $POD -c cassandra -- \
                du -sh /var/lib/cassandra/data/hints/$UUID 2>/dev/null | awk '{print $1}')
            echo "  STALE: $UUID ($SIZE)"
        fi
    done
done
EOF

chmod +x /tmp/find_stale_hints.sh
/tmp/find_stale_hints.sh
```

### Expected Output:

```
Checking cluster1-dc1-rack1-0...
  STALE: bbbb-2222-cccc-3333-dddd-444455556666 (5.2M) # Old Node 101
  STALE: cccc-3333-dddd-4444-eeee-555566667777 (4.1M) # Node 102

Checking cluster1-dc1-rack1-3...
  STALE: bbbb-2222-cccc-3333-dddd-444455556666 (3.8M)
  STALE: cccc-3333-dddd-4444-eeee-555566667777 (6.2M)
```

## Step 3: Delete Stale Hints

```
# Delete stale hints for old Node 101
OLD_NODE_101_UUID="bbbb-2222-cccc-3333-dddd-444455556666"

for i in 0 3 4 5; do
    echo "Cleaning stale hints on cluster1-dc1-rack1-$i..."
    kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
        rm -rf /var/lib/cassandra/data/hints/$OLD_NODE_101_UUID
done

# Delete stale hints for Node 102
NODE_102_UUID="cccc-3333-dddd-4444-eeee-555566667777"

for i in 0 3 4 5; do
    echo "Cleaning stale hints on cluster1-dc1-rack1-$i..."
    kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
        rm -rf /var/lib/cassandra/data/hints/$NODE_102_UUID
done
```

bash

## Step 4: Verify Cleanup

```
# Verify hints removed
/tmp/find_stale_hints.sh
```

bash

**Expected:** No stale hints found

## Method 2: Remove Dead Node from Cluster (Proper Way)

### Step 1: Remove Node from Cluster

```
# Use removemode to properly remove the dead node
kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- \
    nodetool removemode $NODE_102_UUID

# Monitor removal progress
kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- \
    nodetool removemode status
```

bash

### Step 2: Verify Node Removed from Gossip

```
kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- nodetool status
```

bash

**Expected:** Node 102 UUID no longer appears

### Step 3: Check if Hints Auto-Cleanup

```
# After removenode, check if hints are cleaned up
for i in 0 3 4 5; do
  kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
    ls -la /var/lib/cassandra/data/hints/$NODE_102_UUID 2>&1
done
```

bash

**Expected:** Hints may still exist (Cassandra doesn't always auto-cleanup)

**If hints persist, manually delete:**

```
for i in 0 3 4 5; do
  kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
    rm -rf /var/lib/cassandra/data/hints/$NODE_102_UUID
done
```

bash

### Method 3: Node Restart (Triggers Cleanup)

```
# Restart each node one at a time
for i in 0 3 4 5; do
  echo "Restarting cluster1-dc1-rack1-$i..."
  kubectl delete pod cluster1-dc1-rack1-$i -n cass-test

  # Wait for pod to be ready
  kubectl wait --for=condition=ready pod/cluster1-dc1-rack1-$i -n cass-test --timeout=300s

  # Wait for node to be UN
  sleep 30

  echo "Checking hints after restart..."
  kubectl exec -n cass-test cluster1-dc1-rack1-$i -c cassandra -- \
    ls /var/lib/cassandra/data/hints/ 2>/dev/null || echo "Hints cleaned"
done
```

bash

**Note:** Restart may trigger cleanup of some stale hints, but not guaranteed for all

### Method 4: Automated Cleanup Script (Production Solution)

#### Create Cleanup CronJob

yaml

```
# Save as stale-hints-cleanup-cronjob.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cassandra-stale-hints-cleanup
  namespace: cass-test
spec:
  schedule: "0 2 * * *" # Daily at 2 AM
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName: cassandra-operator
          containers:
            - name: cleanup
              image: bitnami/kubectl:latest
              command:
                - /bin/bash
                - -c
                - |
                  #!/bin/bash
                  set -e

                  echo "Starting stale hints cleanup..."

                  # Get current cluster UUIDs
                  CURRENT_UUIDS=$(kubectl exec -n cass-test cluster1-dc1-rack1-0 -c cassandra -- \
                    nodetool status | awk '/^[UD][NLJ]/ {print $7}')

                  # Check each pod
                  for POD in $(kubectl get pods -n cass-test -l app=cassandra -o name); do
                    POD_NAME=$(basename $POD)
                    echo "Checking $POD_NAME..."

                    # Get hint directories
                    HINT_DIRS=$(kubectl exec -n cass-test $POD_NAME -c cassandra -- \
                      ls /var/lib/cassandra/data/hints/ 2>/dev/null || echo "")

                    for UUID in $HINT_DIRS; do
                      # Check if UUID is stale
                      if ! echo "$CURRENT_UUIDS" | grep -q "$UUID"; then
                        SIZE=$(kubectl exec -n cass-test $POD_NAME -c cassandra -- \
                          du -sh /var/lib/cassandra/data/hints/$UUID 2>/dev/null | awk '{print $1}')
                        echo "  Removing stale hints: $UUID ($SIZE)"
                        kubectl exec -n cass-test $POD_NAME -c cassandra -- \
                          rm -rf /var/lib/cassandra/data/hints/$UUID
                      fi
                    done
                  done

                  echo "Cleanup complete"
                  restartPolicy: OnFailure
```

## Deploy CronJob

```
kubectl apply -f stale-hints-cleanup-cronjob.yaml bash  
  
# Test run manually  
kubectl create job --from=cronjob/cassandra-stale-hints-cleanup test-cleanup -n cass-test  
  
# Check logs  
kubectl logs -n cass-test job/test-cleanup
```

## Method 5: Mission Control Automation (If Available)

```
# Using Mission Control API to create cleanup workflow bash  
curl -X POST "https://mission-control-url/api/v1/workflows" \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "name": "stale-hints-cleanup",  
  "schedule": "0 2 * * *",  
  "clusters": ["cluster1"],  
  "tasks": [  
    {  
      "type": "execute_script",  
      "script": "find_and_remove_stale_hints.sh"  
    }  
  ]  
'
```

## Production Recommendations

### Immediate Actions

1. **Audit Existing Clusters:** Run stale hints detection script on all production clusters
2. **Clean Up Stale Hints:** Remove identified stale hints to reclaim disk space
3. **Document Node Operations:** Record UUIDs during all node replacement operations

### Short-term Solutions

1. **Manual Cleanup Process:** Establish runbook for post-replacement cleanup
2. **Monitoring:** Add alerts for hint directory disk usage
3. **Node Removal:** Always run `nodetool removenode` for permanently failed nodes



## Long-term Solutions

1. **Automated Cleanup:** Deploy CronJob for regular stale hints cleanup
2. **Mission Control Integration:** Use workflows for centralized automation
3. **Operational Procedures:** Update node replacement procedures to include hint cleanup
4. **Capacity Planning:** Factor hint storage into disk capacity planning

## Best Practices

1.  Always remove dead nodes properly with `nodetool removenode`
2.  After node replacement, manually clean old UUID hints
3.  Implement automated stale hints monitoring
4.  Set alerts for hint directory disk usage (>10GB warning, >50GB critical)
5.  Document node UUIDs during operations
6.  Regular audits of hint directories (weekly/monthly)
7.  Include hint cleanup in node replacement runbooks

## Monitoring Metrics

Monitor these metrics to detect stale hints issues:

- **Disk usage:** `/var/lib/cassandra/data/hints/` directory size
- **Hint count:** Number of hint files per node
- **Hint age:** Age of oldest hint file
- **Failed deliveries:** Log entries for failed hint delivery attempts

## Test Validation Checklist

---

### Phase 1: Initial State

- ☐ All 6 nodes UP and healthy
- ☐ No existing hints
- ☐ Test keyspace and table created

### Phase 2: Node Failures

- ☐ Node 101 and 102 marked as DOWN
- ☐ Hints created on coordinator nodes
- ☐ Hint directories named with correct UUIDs
- ☐ Hint files contain data

### Phase 3: Node 101 Replacement (Within Hint Window)

- ☐ Replacement performed within 3 hours of failure
- ☐ New node has different UUID
- ☐ Old hints for Node 101 UUID still exist (stale)
- ☐ Logs show failed delivery attempts
- ☐ New node is UP and streaming data
- ☐ Stale hints remain on disk requiring cleanup

### Phase 3B: Node Replacement After Hint Window (Alternative)

- ☐ Wait 3+ hours after node failure
- ☐ Verify NO NEW hints created after window expires
- ☐ Old hints remain frozen at pre-window size
- ☐ Replacement performed after hint window
- ☐ New node has different UUID
- ☐ Old hints STILL present (not auto-cleaned)
- ☐ Comparison: Both scenarios require manual cleanup

### Phase 4: Node 102 Long-term Down

- ☐ After hint window: no NEW hints created
- ☐ Old hints persist indefinitely
- ☐ Disk space consumed by stale hints
- ☐ Node still in gossip as DOWN

### Phase 5: Cleanup Validation

- ☐ Stale hints identified correctly
- ☐ Manual deletion successful

- ☐ Disk space reclaimed
- ☐ No impact on cluster operations
- ☐ Automated cleanup working (if implemented)

## Critical Validation: Hint Window Behavior

- ☐ Within window: New hints continue to accumulate
- ☐ After window: No new hints created
- ☐ Both cases: Old hints persist indefinitely
- ☐ Confirmed: Hint window does NOT trigger automatic cleanup

## Summary

---

### Key Findings

1. **No Automatic Cleanup:** Cassandra does NOT automatically remove hints for replaced or permanently down nodes
2. **Disk Space Risk:** Stale hints accumulate indefinitely, potentially causing disk exhaustion
3. **Operational Confusion:** Metrics and logs show pending hints that will never deliver
4. **Hint Window Behavior:**
  - **Within window:** New hints continue to accumulate for down nodes
  - **After window:** No NEW hints created, but OLD hints persist
  - **Critical:** Hint window does NOT trigger automatic cleanup of existing hints
5. **Multiple Scenarios:**
  - **Replaced node (within window):** Old UUID hints become stale, new hints may still be created until replacement
  - **Replaced node (after window):** Old UUID hints remain frozen, no new hints created
  - **Permanently down node:** Hints persist after hint window expires
  - **Removed node:** Hints may persist even after `removenode`
  - **All scenarios:** Manual cleanup required

## Critical Operational Gap

**Cassandra has no built-in mechanism to clean up hints for nodes that are permanently gone but not formally removed from the cluster.**

## Required Actions

- Implement manual or automated cleanup procedures
- Monitor hint directories for stale UUIDs
- Always properly remove dead nodes from cluster
- Include hint cleanup in operational runbooks

## References

---

- [Cassandra Hinted Handoff Documentation](#)
- [nodetool removemode](#)
- [Kubernetes StatefulSets](#)
- [cass-operator Documentation](#)

*Document created: 2026-05-19*

*Last updated: 2026-05-19*

*Version: 1.0*